Vidarbha Youth Welfare Society's

Prof. Ram Meghe Institute of Technology & Research, Badnera-Amravati (M.S.) 444701

(An Autonomous Institute)

Laboratory Manual

Semester IV

Subject: -Digital Communication System & Signal Processing- Lab

(Subject code: - 4SX01)

(Academic Year: 2025-2026)

Department of Industrial Internet of Things(IIoT)
A+ Accredited by NAAC
Phone: (0721) -2681246, Fax No. 0721 – 2681337
Website: www.mitra.ac.in

Vision and Mission of Institute and Department

Vision of the Institute

To become a pace-setting Centre of excellence believing in three universal values namely Synergy, Trust and Passion, with zeal to serve the Nation in the global scenario.

Mission of the Institute

M1: To achieve the highest standard technical education through state-of-the-art pedagogy and enhanced industry institute linkage.
M2: To inculcate the culture of research in core and emerging areas.
M3: To strive for overall development of studies so as to nurture ingenious technocrats as well as responsible citizens.

Vision and Mission of Department of Computer Science and Engineering (IOT)

Vision of the Department

To achieve excellence in the field of Computer Science & Engineering and Internet of Things through dedication & collaboration, so as to meet global standards in education, research and industrial applications and for nurturing young professionals & entrepreneurs with strong belief in social & ethical values for progress of the society.

Mission of the Department

M1: To provide state of the art facilities in order to achieve excellence in teaching – learning, innovative activities, entrepreneurship and research.
M2: To frame curricula and syllabi based on technological needs of industry along with due consideration for moral, ethical and social values, for nation building.
M3: To encourage inter-disciplinary projects and research through industry- institute linkages so as to bridge the gap between industry and academia.
M4: To develop technology-based solutions for relevant societal problems such as environment, health, etc. and impart experiential learning through inter disciplinary/ collaborative research.

Program educational objective (PEO's), program outcomes (PO's) and Program Specific Outcomes (PSO's)

Program educational objective (PEO's)

PEO1: Provide students with opportunities to acquire the ability to apply the fundamentals of mathematics, sciences and engineering to quantitatively analyze engineering problems.

PEO2: Provide students with opportunities to develop industry oriented innovative design skills, including the ability to formulate problems, to think creatively, to synthesize information, and to communicate effectively.

PEO3: Provide students with opportunities to develop the ability to use modern experimental techniques; collect, analyze, and interpret experimental data; and effectively communicate the results.

PEO4: Provide opportunities to prepare students with the diverse skills needed to be successful and competent engineers and/ or to pursue higher studies.

Program Outcomes (PO's)

Engineering Graduate will be able to:

PO1. Engineering knowledge: Apply the knowledge of mathematics, science, engineering fundamentals, and an engineering specialization to the solution of complex engineering problems.

PO2. Problem analysis: Identify, formulate, review research literature, and analyse complex engineering problems reaching substantiated conclusions using first principles of mathematics, natural sciences, and engineering sciences.

PO3. Design/development of solutions: Design solutions for complex engineering problems and design system components or processes that meet the specified needs with appropriate consideration for the public health and safety, and the cultural, societal, and environmental considerations.

PO4. Conduct investigations of complex problems: Use research-based knowledge and research methods including design of experiments, analysis and interpretation of data, and synthesis of the information to provide valid conclusions.

PO5. Modern tool usage: Create, select, and apply appropriate techniques, resources, and modern engineering and IT tools including prediction and modelling to complex engineering activities with an understanding of the limitations.

PO6. The engineer and society: Apply reasoning informed by the contextual knowledge to assess societal, health, safety, legal and cultural issues and the consequent responsibilities relevant to the professional engineering practice.

PO7. Environment and sustainability: Understand the impact of the professional engineering solutions in societal and environmental contexts, and demonstrate the knowledge of, and need for sustainable development.

PO8. Ethics: Apply ethical principles and commit to professional ethics and responsibilities and norms of the engineering practice.

PO9. Individual and team work: Function effectively as an individual, and as a member or leader in diverse teams, and in multidisciplinary settings.

PO10. Communication: Communicate effectively on complex engineering activities with the engineering community and with society at large, such as, being able to comprehend and write effective reports and design documentation, make effective presentations, and give and receive clear instructions.

PO11. Project management and finance: Demonstrate knowledge and understanding of the engineering and management principles and apply these to one's own work, as a member and leader in a team, to manage projects and in multidisciplinary environments.

PO12. Life-long learning: Recognize the need for, and have the preparation and ability to engage in independent and life-long learning in the broadest context of technological change.

Program Specific Outcomes (PSO's)

PSO1: Develop different problem-solving skills in order to solve industrial problems based on principles and concepts of CSE & IoT.

PSO2: Comprehend various application development tools and their implementation for IoT applications.

PSO3: To design and implement CSE and IoT based solution for technology based industrial problems

1. Communication and Industrial Network Lab (Subject code: - 4SX01)

Course Number and Title:	Digital Communication System & Signal Processing- Lab
Credits:	3(Th) + 1(Lab)
Name of Faculty:	Dr.K.S.Belsare,Prof.A.S.Chaudhari,Prof.P.M.Dundale,Prof.D.P.Paraskar
Course Type:	Theory + Lab
Compulsory/ Elective:	Compulsory
Teaching Methods:	Lecture: 3 Hrs/ week. Laboratory: 2 Hrs/ week.
Course Assessment: (Practical)	Laboratory: Lab Performance & Final Submission by Internal + Viva-Voce Semester End External Examination Practical: 50 % Internal (25 Marks) + 50 % External (25 Marks).
Grading Policy:	2. Grading Table

Grades	Marks
A+	10
A	9
B+	8
B	7
C+	6
C	5
D	4

	Subject: Digital Communication System & Signal Processing	L
Unit-I	**Digital Pulse Code Modulation:** Digital representation of Analog signal, PCM Generation, Uniform quantization (mid-rise and mid-tread), Quantization Noise and Quantization error, Companding, Differential Pulse Code Modulation, Delta Modulation, Adaptive Delta Modulation	07
Unit-II	**Introduction to Information Theory and Digital Communication System:** Information Theory: Measure of Information, Entropy and Information Rate, Huffman Coding, Binary Symmetric Channel Digital Communication System: Functional Blocks of Digital Communication System, Line Coding: Need for Line coding, Types of Line Coding.	07
Unit-III	**Digital Modulation:** ASK, BPSK and BFSK generation and reception, Introduction to Probability of Error, QPSK and MSK Transmitter and Receiver, ISI, Need for equalization , 3-stage tap equalizer	07
Unit-IV	**Discrete Time Signals and Systems:** Sampling theorem, Proof of Sampling Theorem, Signal reconstruction, DT Signal Operations, DT Signal Classification, Energy and Power Signal, Even and Odd functions, DT Convolution, Classification of Discrete-Time Systems.	07
Unit-V	**Discrete-Time Fourier Transform:** Discrete-Time Fourier Transform (DTFT), Properties of DTFT, Relationship between DTFT & CTFT, Introduction to DFT	07

Unit-VI	**Z-Transform:** Definition and Properties of Z-Transform, concept of Region of Convergence [ROC], Inverse Z-transform using long division method, PFE method and residue method.	07
	Total	**42**

Text Books:

1. Shanmugam K.S., "Digital & Analog Communication Systems", John Wiley & Sons, NewYork, 1996.
2. Simon Haykin, "Digital Communication", John Wiley & Sons, Pvt. Ltd. Singapore.
3. A Nagoorkani, "Signal and Systems", Tata McGraw Hill Education Pvt. Ltd. 2010.
4. P. Ramesh Babu, R. Anandanatrajan, "Signals and Systems", Scitech Publications, Chennai, Fourth Edition, 2011.

References:

1. Proakis J. K., "Digital Communication", Mc-Graw Hill Book Co., London (2nd Edition).
2. Taub and Schilling D.L., "Principles of Communication Systems", Mc-Graw Hill Co., New Delhi (2nd Ed.).
3. B. P. Lathi, "Modern Digital and Communication Systems", Holt Rinchart and Winston Inc,1993.
4. Alan V. Oppenheim & Alan S. Willsky with S. Hamid Nawab, "Signals & Systems" PHI publications, Second Edition.
5. Lathi B. P., "Principles of Linear Systems and Signals" Second Edition (International Version) Oxford University Press.
6. Ambardar A., "Analog and Digital Signal Processing", Thomson Learning-2005

B.Tech Third Semester CSE(IoT)

Course Learning Objectives:

1. To understand the practical approach to digital communication & signal processing.

2. To learn and understand modulation and demodulation techniques.

3. To learn the realization and able to understand the digital communication and Multi-rate processing.

Course Outcomes:

After successful completion of this course, students will be able to-

Sr. No	CO ID	Description
1	4SX06.1	Use his/her understanding/knowledge of digital communication so as to design /conduct experimentation related to the given aim/objective.
2	4SX06.2	Use equipment's/other concerned resources for conduction of experiments.
3	4SX06.3	Interpret results, formulate findings, draw suitable conclusion and respond to inquiries, related to given experiment

Mapping of Course Outcomes with Program Outcomes

Course	PO1	PO2	PO5	PO9	PO10
4SX06.1					
4SX06.2					
4SX06.3					

Correlation levels: 1. Low 2. Medium 3. High

Mapping of Course Outcomes with Program Specific Outcomes

Course	PSO 1	PSO 2	PSO 3
4SX06.1			
4SX06.2			
4SX06.3			

Correlation levels: 1. Low 2. Medium 3. High

4. <u>Gradation Policy for Laboratory Assessment of each Practical for B.Tech Programme</u>

The practicals are of three types –

1. That involves lab equipment such as oscilloscopes, component development systems, signal generators (Test bench)
2. That involves Computers (Simulation)
3. That involves Field work (Community Engagement Project/ Field Project (CEP/FP), mini project, capstone projects etc.)

Following are the components involved in the gradation of the practicals except Community Engagement Project/ Field Project (CEP/FP), mini project, capstone projects etc. by the respective teachers

P1: Attempt and/or attention for execution of Practical task.

P2: Competency (Skill/ability) for performing particular task.

P3: Performance and timely submission of Practical records

P4: Ability to analyze results and draw conclusions.

P5: Question and Answers / Viva-Voce.

Grade	P1	P2	P3	P4	P5
A+	Performance in First attempt; exceptional attention to task	Able to apply skill effectively	Excellent performance & submission of practical record along with results and conclusion by consecutive practical day.	Highly insightful analysis with accurate, logical, and well-developed conclusions	Appropriate and accurate answers
A	Performance in First attempt; excellent attention to task	Good skill application	Effective performance & submission of practical record along with results and conclusion by consecutive practical day.	Clear, accurate analysis with well-reasoned conclusions	Relevant and well-explained answers
B+	Performance in First attempt; above average attention to task	Satisfactory skill application	Good performance & submission of practical record one week late then timely date of submission	Generally correct analysis with mostly appropriate conclusions	Mostly appropriate answers
B	Performance in second attempt; average attention to task	Minimal understanding skill application	Acceptable performance & submission of practical record one week late then timely	Minimal analysis with partially correct or incomplete	Partially appropriate answers

			date of submission	conclusions	
C+	Performance in second attempt; below average attention to task	Somewhat skill application	Inconsistent performance & submission of practical record two week late then timely date of submission	Limited analysis with conclusions lacking clarity or accuracy.	Limited but somewhat relevant answers
C	Performance in third attempt or more; somewhat attentive	Severe difficulty in applying skills	Poor performance & submission of practical record two week late then timely date of submission	Minimal analysis with weak or unsupported conclusions	Weak or incomplete answers
D	Performance in third attempt or more; least attentive	Able to apply skills with help from faculty	Very poor performance & very late submission of practical record	Unable to analyze results; conclusions incorrect or missing	Incorrect or irrelevant answers

Mapping of Practical's gradation with marks

Grades	Marks
A+	10
A	9
B+	8
B	7
C+	6
C	5
D	4

List of Experiments

Department of CSE-IoT

Subject: - Digital Communication System & Signal Processing-Lab

Semester: - 4th

Subject Code: - 4SX06

Exp. No.	Aim of experiment
1.	To verify the operation of Pulse Code Modulation (PCM) and Demodulation
2.	To analyze and compare performance of Quadrature Phase Shift Keying (QPSK)
3.	To study and perform Amplitude Shift Keying (ASK) modulation and demodulation using an ASK trainer kit and observe the waveforms on CRO/DSO
4.	To perform MSK modulation using the MSK Modulation & Demodulation Trainer Kit
5.	Generation and realization of different Line Codding schemes. Unipolar (RZ and NRZ), Polar (RZ and NRZ), Bipolar NRZ and Manchester codding scheme
6.	To perform Binary Phase Shift Keying (BPSK) modulation and demodulation using a BPSK trainer kit and observe the corresponding waveforms
7.	Develop a program to generate the plot for various basic signals in signal processing
8.	Develop a program for the convolution of two discrete time signals using the Matrix method and validate the result using an in-built convolution command
9.	To develop a program to determine the linearity and time invariance properties of a discrete-time system
10.	Develop program to determine the Fourier Transform of Discrete Time Signal and to plot magnitude and Phase response
11.	Develop a program to verify the following properties of DTFT i)Time Shifting Property ii) Frequency Shifting Property
12.	Develop a program to determine Z-Transform of a Discrete Time signal.

Prof. R. P. Sawarkar

Lab-Incharge

Dr. K. S. Belsare, Prof. A. S. Chaudhari,
Prof. P. M. Dundale, Prof. D. P. Paraskar

Subject Incharge

EXPERIMENT NO. 1

AIM: To verify the operation of Pulse Code Modulation (PCM) and Demodulation.

Apparatus Required: PCM transmitter trainer, PCM receiver trainer, CRO and connecting wires.

Theory: Pulse-code modulation (PCM) is used to convert analog signals into binary digital form. Analog methods suffer from transmission distortion and noise to an appreciable extent, and are therefore not used that often for direct transmission, but more often an intermediate step in the generation of PCM. In the PCM system, groups of pulses or codes are transmitted which represent binary numbers corresponding to modulating voltage levels. Recovery of the trans- mitted information does not depend on the height, width, or energy content of the individual pulses, but only on their presence or absence. since it is relatively easy to recover pulses under these conditions, even in the presence of large amounts of noise and distortion, PCM systems tend to be very immune to interference and noise. Regeneration of the pulses enroute is also relatively easy, resulting in a system that produces excellent results for long distance communication.

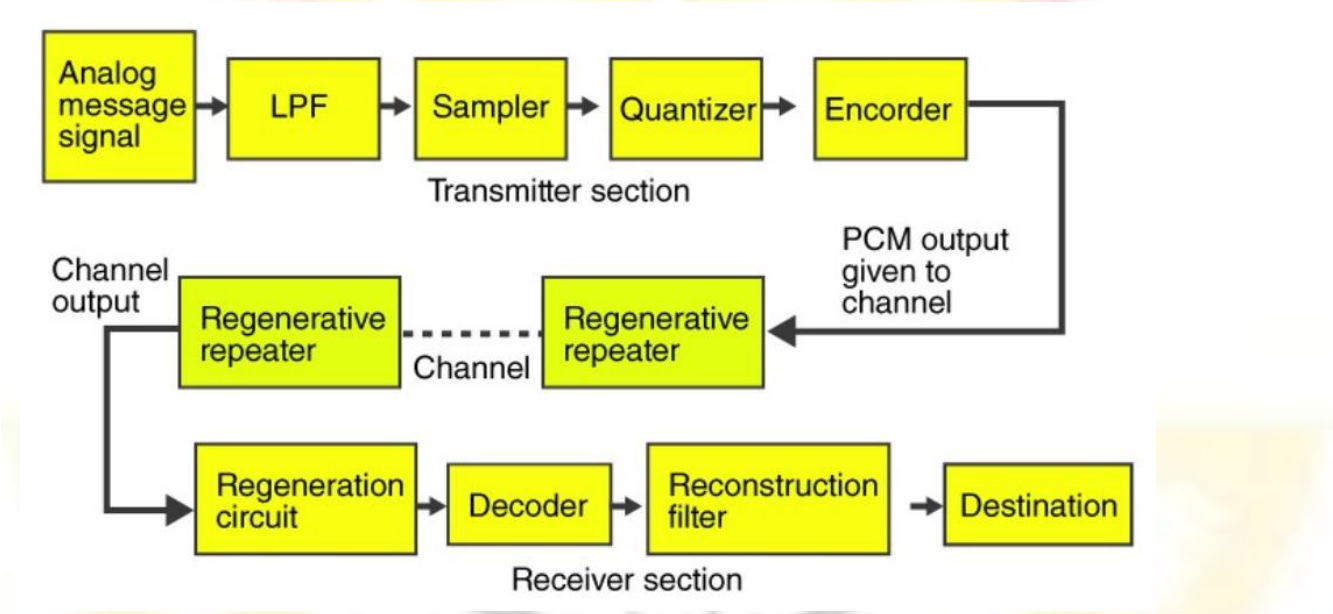


Figure 1.1. Block diagram of PCM System

PROCEDURE:

1. Connect DC input signal at Vin of ADC.

2. Make power on to the Transmitter.

3. Vary the DC input and note the output of ADC. (8-bit word is buffered and displayed by LED on panel).

4. Connect Dx, CLKx, & FSx of the transmitter to DR, CLKR & FSR correspondingly to PCM receiver.

5. Connect Ground of Transmitter to Ground of Receiver.

5. Observe that serially transmitted data is accepted & faithfully converted in 8-bit data word in receiver.

6. Observe the DAC output by measuring average voltage at the output by digital multimeter.

7. Vary the input DC level at input of ADC in transmitter & note the DAC output at receiver.

8. Take at least 5 readings by varying the DC input level.

9. Take at least 5 different readings by varying input to ADC of Transmitter.

10. Connect Analog input signal at the modulating input of S/H Logic, Connect S/H output to Vin of ADC.

11. Observe the various waveforms on CRO at

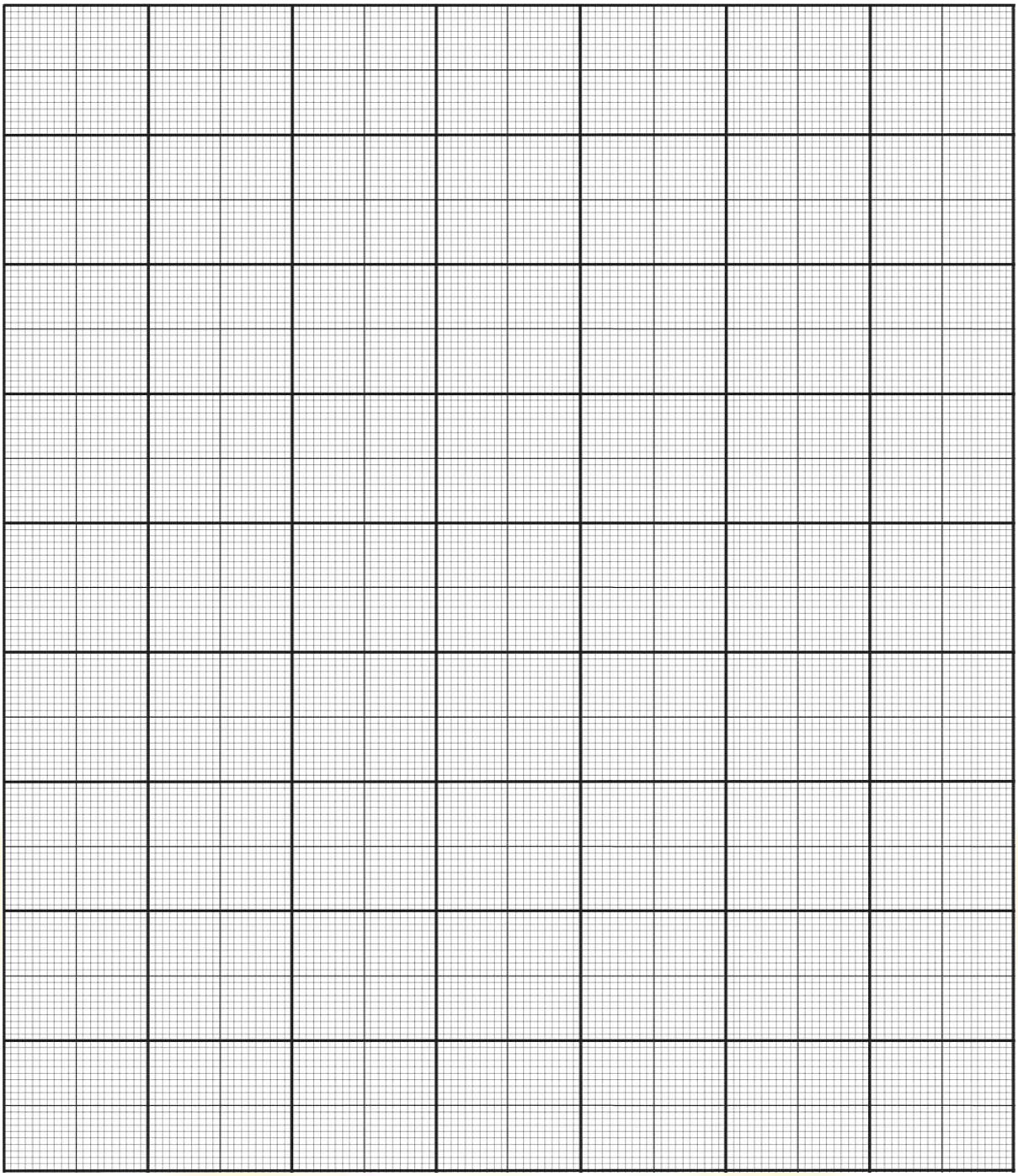
- S/H output in Transmitter
- PCM output in Transmitter
- DAC output in Receiver.
- Reconstructed & filtered, demodulated waveform in Receiver.

12. Vary the amplitude of input analog signal and observe the demodulated output.

Sr. No.	**I/P To ADC Vin (Volt)**	**Digital Data**	
		MSB	**LSB**
		D7 D6 D5 D4	**D3 D2 D1 D0**

Graph: -PCM Waveform

Scale
On X-axis 1cm =
On Y-axis 1cm =



RESULT:

CONCLUSION:

VIVA QUESTIONS:

1) What is the expression for transmission bandwidth in a PCM system?
2) What is the expression for quantization noise /error in PCM system?
3) What are the applications of PCM?
4) What are the advantages of the PCM?
5) What is companding?

EXPERIMENT NO. 2

AIM: To analyze and compare performance of Quadrature Phase Shift Keying (QPSK)

APPARATUS: ST2170A Clock and Carrier Generator, ST2170B Data Source, ST2170E Double Balance Modulator, 4.2 mm Banana cable, Oscilloscope, Scientech 2808 trainer kit, 2.2 mm Banana cable

B. Quadrature Phase Shift Keying (QPSK)

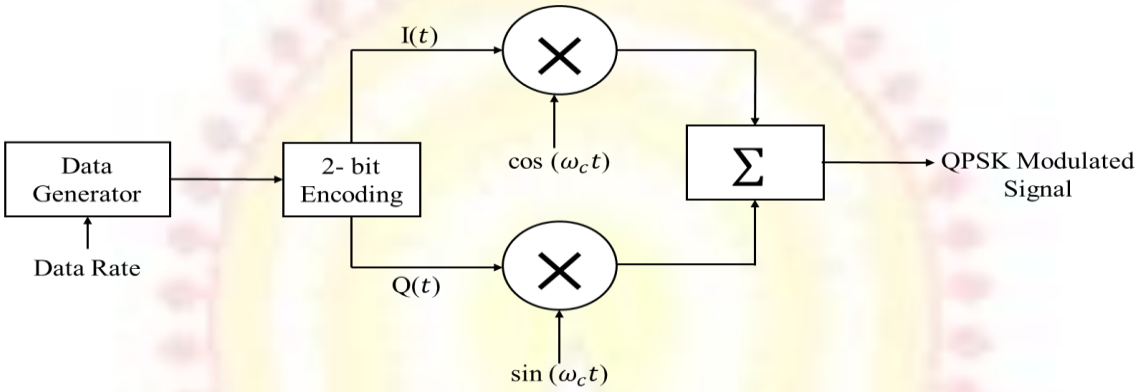


Figure 2.1 QPSK Modulator

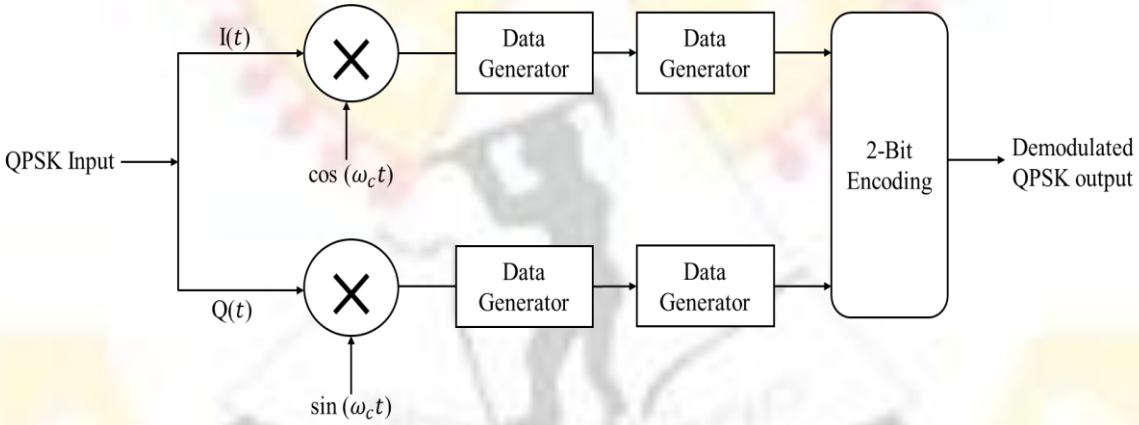


Figure 2.2 QPSK demodulator

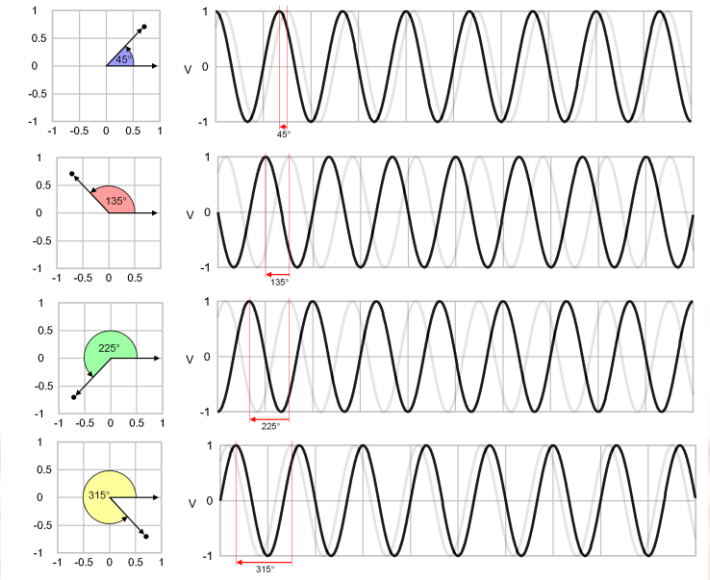


Figure 2.3 Output of QPSK

Procedure for QPSK Modulation:

1. Select QPSK modulator using push button indicated by LEDs.

2. Select the data rate using push button i. e. 2KHz, 4Khz, 8Khz & 16KHz. Observe the change in frequency on Test point (TP1).

3. Select the data bit i.e. 8-bit, 16-bit, 32 bit & 64 bits.

4. Observe the data rate at TP1 by varying data rate using push buttons.

5. Observe the input data bit pattern at TP2 by varying bit pattern.

6. Observe the two-bit encoding at TP3 & TP4.

7. Observe the carrier signal at TP5 & TP6.

8. Observe the I-channel & Q-channel modulated signal at TP7 & TP8 respectively.

9. Observe the QPSK modulated signal at TP9 (refer fig).

Procedure for QPSK Demodulation:

1. Selected QPSK demodulator t indicated by glowing LEDs.

2. Select the data rate using push button i, e. 2KHz, 4Khz, 8Khz & 16KHz.

3. Select the data bit i.e. 8 bit, 16-bit, 32 bit & 64 bit.

4. Observe the multiplied signal of QPSK & cosine carrier signal at TP12, QPSK & sine carrier signal at TP13.

5. Observe the integrated output of I- Channel & Q-Channel at TP14 & TP15 respectively.

6. Observe the comparator output of I-channel & Q-channel at TP16 & TP17 respectively.

7.Observe the QPSK demodulated output signal at TP18 (refer fig d.)

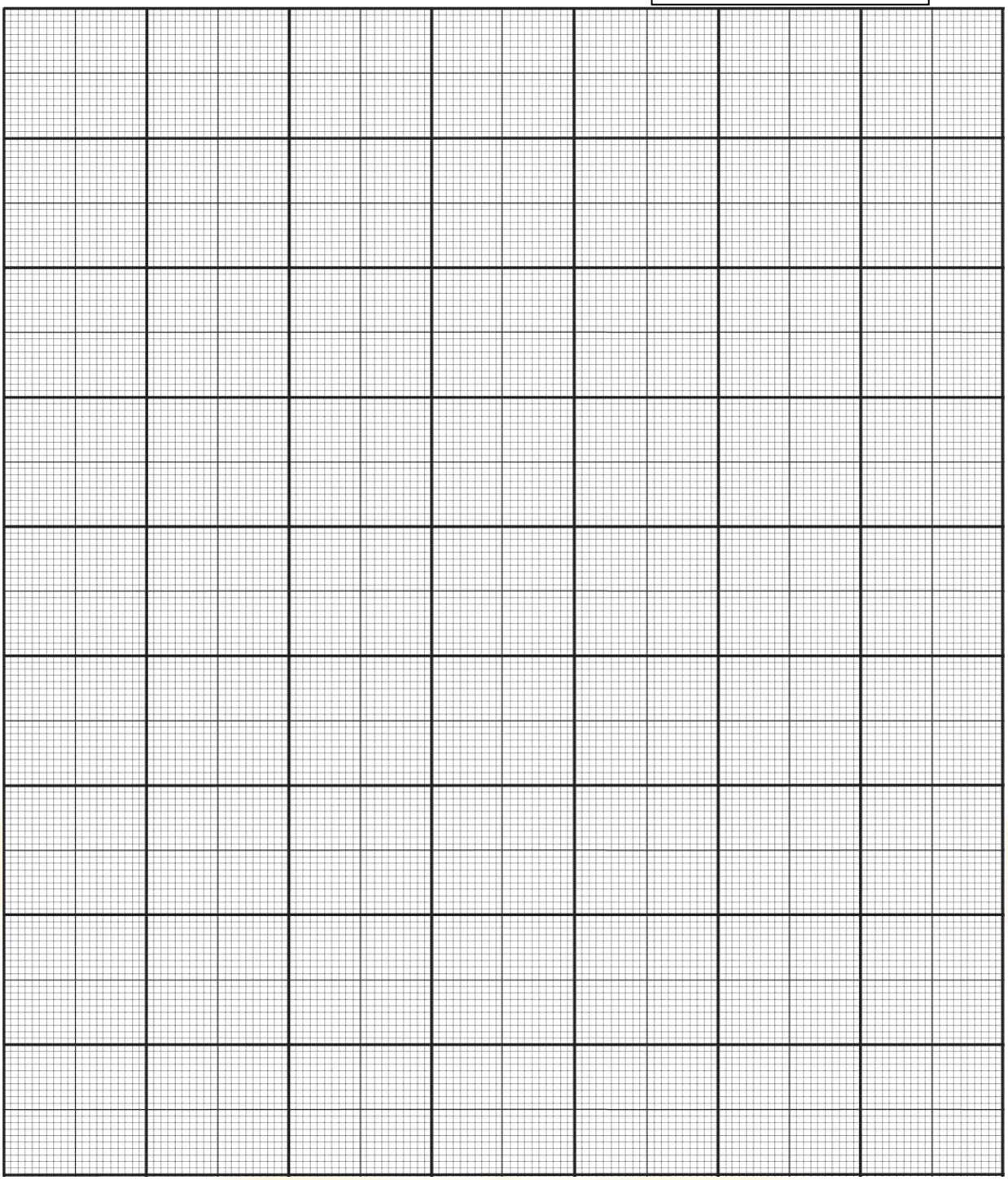
Observation Table:

Sr. No.	QPSK Modulation	Data Bit	Data Rate	Signal Amplitude		Tb		QPSK Signal	
				I-Channel	Q-Channel	I-Channel	Q-Channel	Amplitude	Tb

Sr. No.	QPSK Demodulation	Data Bit	Data Rate	Signal Amplitude		Tb		QPSK Signal	
				I-Channel	Q-Channel	I-Channel	Q-Channel	Amplitude	Tb

Graph: -QPSK Signal

Scale
On X-axis 1cm =
On Y-axis 1cm =



RESULT:

CONCLUSION:

VIVA QUESTIONS:

1. Define QPSK.
2. What is the role of the mixers in the QPSK modulator?
3. How are the signals at the outputs of the mixers combined to produce the QPSK signal?
4. Give the applications of QPSK.

EXPERIMENT NO. 3

AIM: To study and perform Amplitude Shift Keying (ASK) modulation and demodulation using an ASK trainer kit and observe the waveforms on CRO/DSO.

Apparatus Required: ASK Modulation & Demodulation Trainer Kit, CRO / DSO, Patch Cords, Regulated Power Supply

Theory:

ASK Modulation

Amplitude Shift Keying (ASK) is a digital modulation technique in which the amplitude of a high-frequency carrier signal is varied according to the binary data, while the carrier frequency and phase remain constant.

1. When the binary data = 1, the carrier is transmitted with maximum amplitude.

2. When the binary data = 0, the carrier amplitude is reduced or made zero.

ASK is widely used in low-cost and low-data-rate communication systems, such as optical communication and RFID systems.

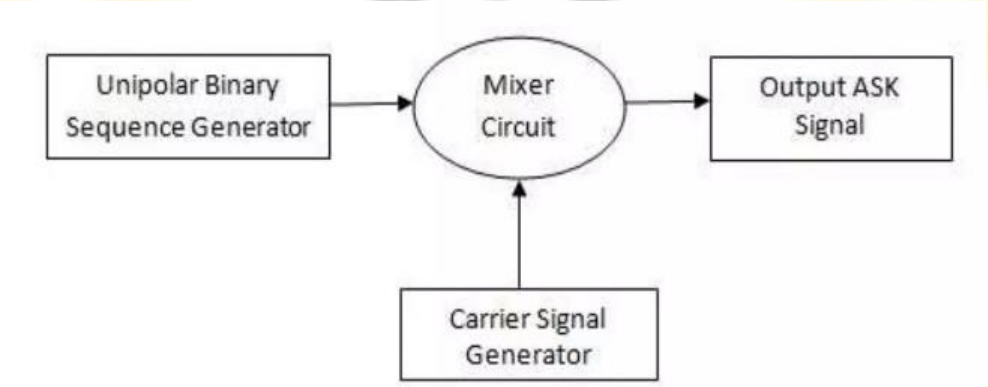
ASK Demodulation

ASK demodulation can be done using an **envelope detector** or **coherent detector.**

Envelope Detection Steps:

1. The received ASK signal is passed through a **rectifier.**
2. A **low-pass filter** extracts the envelope.
3. A **comparator** converts the envelope into binary data.

Block Diagram:



Procedure:

A. ASK Modulation

1. Switch **ON** the power supply of the ASK trainer kit.
2. Set the **data generator** on the kit to generate a binary data signal.
3. Connect:
   - Data output → ASK modulator input
   - Carrier output → ASK modulator carrier input
4. Connect the **ASK modulator output** to CRO Channel-1.
5. Observe:
   - Carrier signal
   - Binary data signal
   - ASK modulated waveform
6. Verify that:
   - Carrier amplitude is **high for binary '1'**
   - Carrier amplitude is **low/zero for binary '0'**

B. ASK Demodulation

1. Connect ASK modulator output to **demodulator input** on the kit.
2. Connect demodulator output to CRO Channel-2.
3. Observe the **demodulated digital signal.**
4. Compare the demodulated output with the original binary input.

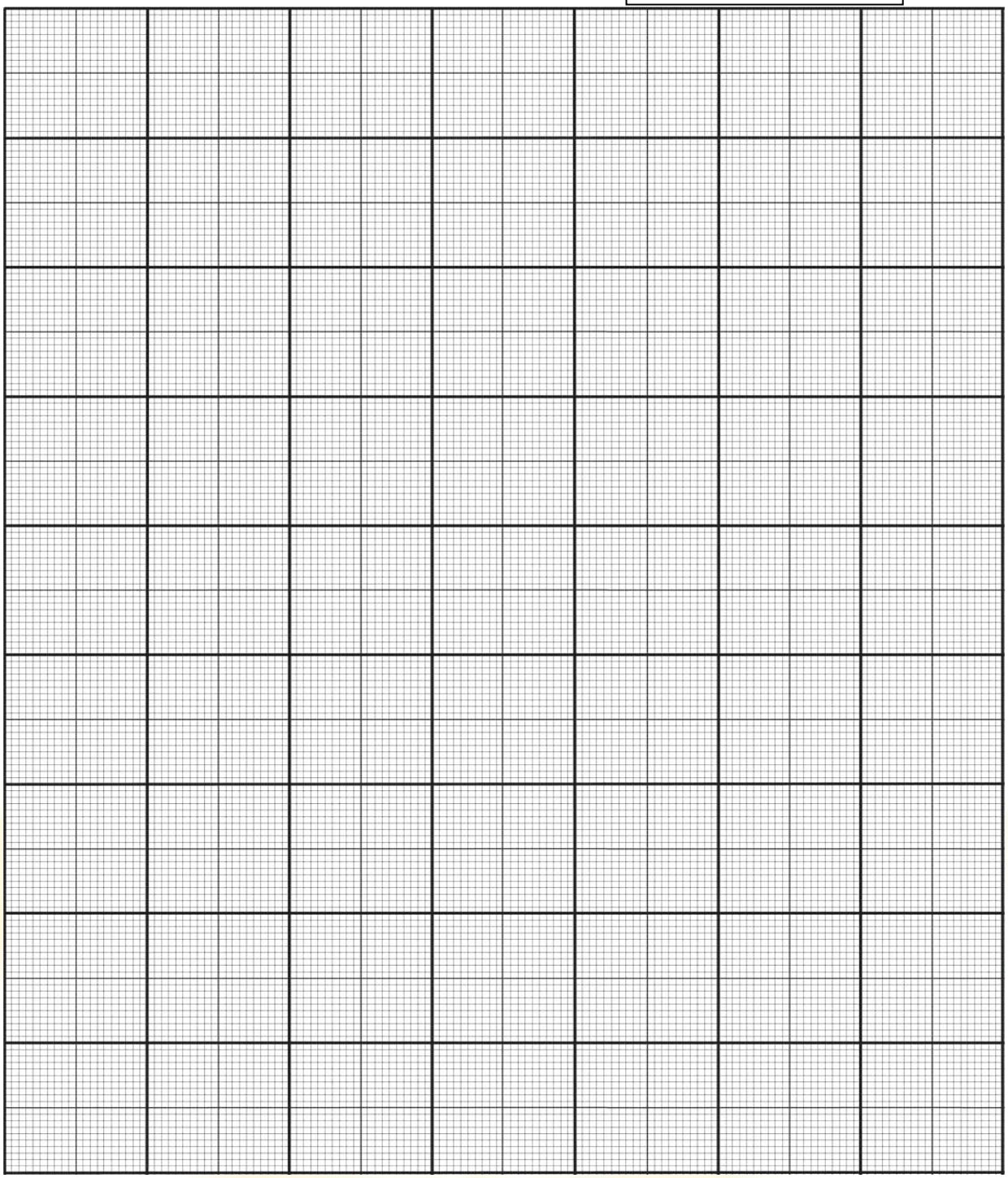
Observation Table:

Sr. No		**Input Bit**	**rrier Amplitude**	**ASK Output**
1	ASK Modulation			
2				
3				
4				

r. No		**ASK Input**	**Demodulated Output**	**Output Bit**
1	ASK Demodulation			
2				
3				
4				

5. **Graph: -ASK Signal**

Scale
On X-axis 1cm =
On Y-axis 1cm =



RESULT:

CONCLUSION:

VIVA QUESTIONS:

1. Why is ASK called On-Off Keying?
2. Which block generates the carrier in the kit?
3. Why is ASK sensitive to noise?
4. What is the role of envelope detector in the kit?
5. How does CRO help in ASK analysis?

EXPERIMENT NO. 4

AIM: To perform MSK modulation using the MSK Modulation & Demodulation Trainer Kit.

Apparatus Required: MSK Modulation & Demodulation Trainer Kit, CRO / DSO, Patch Cords, Regulated Power Supply

Theory: MSK (Minimum Shift Keying) is a continuous phase FSK scheme where the frequency separation between logic '1' and logic '0' is minimum and equal to half the bit rate. It provides constant envelope**,** continuous phase**,** and better spectral efficiency compared to conventional FSK.

Procedure:

A. MSK Modulation

1. Switch ON the MSK Trainer Kit.
2. Select internal clock = 16 kHz**.**
3. Select internal data generator (NRZ)**.**
4. Connect patch cords as per MSK modulator block diagram printed on the kit**.**
5. Observe waveforms at different test points using CRO.
6. Note amplitude and frequency values.
7. Observe MSK output waveform.

B. MSK Demodulation

1. Connect MSK modulator output → MSK demodulator input**.**
2. Connect demodulator section as per block diagram.
3. Observe LPF outputs, integrator outputs, and decision circuit output.
4. Compare transmitted and received data.

Observation Table:

Sr. No		**Signal Observed**	**Frequency**	**Amplitude**
1	MSK Modulation			
2				
3				
4				

1. **Graph: -MSK Signal**

Scale
On X-axis 1cm =
On Y-axis 1cm =

RESULT:

CONCLUSION:

VIVA QUESTIONS:

1. What is Minimum Shift Keying (MSK)?
2. Why is MSK called a continuous phase modulation technique?
3. What is the frequency separation used in MSK?
4. How is MSK related to OQPSK?
5. Why is MSK preferred in wireless communication systems?

EXPERIMENT NO. 5

AIM: Generation and realization of different Line Codding schemes. Unipolar (RZ and NRZ), Polar (RZ and NRZ), Bipolar NRZ and Manchester codding scheme.

Simulator: - GNU Octave

Theory: UNIPOLAR LINE CODING: -

Line coding is the process of converting digital data to digital signals. In Unipolar line coding all the signal levels are either above or below the axis. It has only one voltage level other than zero. The symbols 0&1 in digital system can represented in various formats with different levels and wave forms. The selection of particular format for common pulse depends on the systems band width, system's ability to pass DC level information, error checking facility, case of clock regeneration & synchronization at receiver, complexity & cost etc.

NON RETURN TO ZERO (LEVEL) NRZ: - It is the simplest form of data representation. The NRZ waveform simply goes low for one bit time to represent a data 0 & high for one bit time to represent a data 1. It is unipolar line coding scheme in which positive voltage defines bit 1 and the zero voltage defines bit 0. Signal does not return to zero at the middle of the bit thus it is called NRZ. RETURN TO ZERO (LEVEL) RZ: - Return-to-zero (RZ or RTZ) describes a line code used in telecommunications signals in which the signal drops (returns) to zero between each pulse. That "zero" condition is typically halfway between the significant condition representing a 1 bit and the other significant condition representing a 0 bit. RZ uses pulses at the start of the clock cycle to indicate a 1 value.

POLAR LINE CODING: -

Line coding is the process of converting digital data to digital signals. In Polar line coding the voltages are on the both sides of the axis. Which means it will have both positive and negative values for voltages or amplitude, it is quite like NRZ scheme but, here we have NRZ-L (i.e., NRZ-Level) and NRZ-I (i.e., NRZInvert).It has two voltage level other than zero. The symbols 0&1 in digital system can represented in various formats with different levels and wave forms. The selection of particular format for common pulse depends on the systems band width, system's ability to pass DC level information, error checking facility, case of clock regeneration & synchronization at receiver, complexity & cost etc. NRZ-L AND NRZ-I – These are somewhat similar to unipolar NRZ scheme but here we use two levels of amplitude (voltages).

For NRZ-L(NRZ-Level), the level of the voltage determines the value of the bit, typically binary 1 maps to logic-level high, and binary 0 maps to logic-level low, and for NRZI(NRZ-Invert), two-level signal has a transition at a boundary if the next bit that we are going to transmit is a logical 1, and does not have a transition if the next bit that we are going to transmit is a logical 0.

RETURN TO ZERO RZ: - One solution to NRZ problem is the RZ scheme, which uses three values positive, negative, and zero. In this scheme signal goes to 0 in the middle of each bit. The logic we are using here to represent data is that for bit 1 half of the signal is represented by +V and half by zero voltage and for bit 0 half of the signal is represented by -V and half by zero voltage.

BIPOLAR LINE CODING: -

In this scheme there are three voltage levels positive, negative, and zero. The voltage level for one data element is at zero, while the voltage level for the other element alternates between positive and negative. Bipolar encoding, binary zero ('0') is represented by 0 voltage whereas binary one ('1) is represented by alternating positive and negative voltages. The Bipolar coding technique uses different types of pulses based on which it is known as NRZ (Non-Return to Zero) or RZ (Return to Zero).

NOT RETURN TO ZERO (NRZ): - Uses three levels of signal level (+A, 0, -A)and has “Alternate Mark Inversion” (AMI).Bipolar NRZ coding uses zero voltage to represent binary zero and +ve pulse and -ve pulse to represent alternating binary ones during entire bit period. Hence pulse duration and symbol bit duration are equal in NRZ type.

RETURN TO ZERO (RZ): - Uses three levels of signal level (+A, 0, -A) has “Alternate Mark Inversion” (AMI). There is a half-width +ve output pulse if the input is a ‘1’; or a half-width -ve output pulse if the input is a ‘0’. There is a return-to-zero for the second half of each bit period.

Manchester coding: Here, in this type of coding, symbol 1 is characterized by transmit a +ve pulse (say +A volts) for one-half of the signal length followed by a -ve pulse (say -A volts) for the other half of the signal length. Correspondingly, symbol ‘0’ is characterized by a -ve half-bit pulse following the +ve half-bit pulse in Manchester encoding techniques.

Code:

```
clc;
clear all;
close all;
A = 1;
B = -1;
msg_seq = input("Enter the sequence of msg in [] ==> ");

L = length(msg_seq);
#---------------------------- Unipolar NRZ -------------------------------#

indexx = 1;
time = 0 ;
Tb = 1;
UNRZ_sarray0 = [0 0 0 0];
UNRZ_sarray1 = [A A A A];
for n = 1:L
if(msg_seq(n)==1)
seq_format(indexx:indexx+3) = UNRZ_sarray1;

else
seq_format(indexx:indexx+3) = UNRZ_sarray0;
endif
time_format(indexx:indexx+3) = [time time+(Tb/2) ...
time+(Tb/2) time+Tb];
indexx = indexx + 4;
time = time + Tb;
endfor
subplot(3,2,1),plot(time_format,seq_format,'r'),grid on;
title("Unipolar NRZ"),xlabel("Time --->"),ylabel("Amplitude");
```

```
axis([0 L+1 -1.5 1.5]);
#-----------------------------------------------------------------------------#
#---------------------------------- Unipolar RZ ----- -----------------------#

indexx = 1;
time = 0 ;
Tb = 1;
URZ_sarray0 = [0 0 0 0];
URZ_sarray1 = [A A 0 0];
for n = 1:L
if(msg_seq(n)==1)
seq_format(indexx:indexx+3) = URZ_sarray1;
else
seq_format(indexx:indexx+3) = URZ_sarray0;
endif
time_format(indexx:indexx+3) = [time time+(Tb/2) time+(Tb/2) time+Tb];
indexx = indexx + 4;
time = time + Tb;
endfor
subplot(3,2,2),plot(time_format,seq_format,'r'),grid on;
title("Unipolar RZ"),xlabel("Time --->"),ylabel("Amplitude");
axis([0 L+1 -1.5 1.5]);
#-----------------------------------------------------------------------------#
#------------------------------- Polar NRZ --------------------------------#
indexx = 1;
time = 0 ;
Tb = 1;
PNRZ_sarray0 = [B B B B];
PNRZ_sarray1 = [A A A A];
for n = 1:L
if(msg_seq(n)==1)
seq_format(indexx:indexx+3) = PNRZ_sarray1;
else
seq_format(indexx:indexx+3) = PNRZ_sarray0;
endif
time_format(indexx:indexx+3) = [time time+(Tb/2) time+(Tb/2) time+Tb];

indexx = indexx + 4;
time = time + Tb;
endfor
subplot(3,2,3),plot(time_format,seq_format,'r'),grid on;
title("polar NRZ"),xlabel("Time --->"),ylabel("Amplitude");
axis([0 L+1 -1.5 1.5]);
#-----------------------------------------------------------------------------#
#------------------------------- Polar RZ ---------------------------------#
indexx = 1;
time = 0 ;
Tb = 1;
PRZ_sarray0 = [B B 0 0];
PRZ_sarray1 = [A A 0 0];
for n = 1:L
```

```
if(msg_seq(n)==1)
seq_format(indexx:indexx+3) = PRZ_sarray1;
else
seq_format(indexx:indexx+3) = PRZ_sarray0;
endif
time_format(indexx:indexx+3) = [time time+(Tb/2) time+(Tb/2) time+Tb];
indexx = indexx + 4;
time = time + Tb;
endfor
subplot(3,2,4),plot(time_format,seq_format,'r'),grid on;
title("polar RZ"),xlabel("Time --->"),ylabel("Amplitude");
axis([0 L+1 -1.5 1.5]);
#-------------------------------------------------------------------------------#
#------------------------------- Bipolar NRZ ----------------------------#
indexx = 1;
time = 0 ;
Tb = 1;
BNRZ_sarray0 = [0 0 0 0];
BNRZ_sarray1 = [A A A A;B B B B];
shift = [1,2];
for n = 1:L
if(msg_seq(n)==0)
seq_format(indexx:indexx+3) = BNRZ_sarray0;
else
seq_format(indexx:indexx+3) = BNRZ_sarray1(shift(1),:);
shift = [shift(2),shift(1)];
endif
time_format(indexx:indexx+3) = [time time+(Tb/2) time+(Tb/2) time+Tb];
indexx = indexx + 4;
time = time + Tb;
endfor
subplot(3,2,5),plot(time_format,seq_format,'r'),grid on;

title("Bipolar NRZ"),xlabel("Time --->"),ylabel("Amplitude");
axis([0 L+1 -1.5 1.5]);
#-------------------------------------------------------------------------------#
#------------------------------- Manchester coding ----------------------------------#
indexx = 1;
time = 0 ;
Tb = 1;
M_sarray0 = [B/2 B/2 A/2 A/2];
M_sarray1 = [A/2 A/2 B/2 B/2];
for n = 1:L
if(msg_seq(n)==1)
seq_format(indexx:indexx+3) = M_sarray1;
else
seq_format(indexx:indexx+3) = M_sarray0;
endif
time_format(indexx:indexx+3) = [time time+(Tb/2) time+(Tb/2) time+Tb];
indexx = indexx + 4;
time = time + Tb;
```

```
endfor
subplot(3,2,6),plot(time_format,seq_format,'r'),grid on;
title("Manchester"),xlabel("Time --->"),ylabel("Amplitude");
axis([0 L+1 -1.5 1.5]);
#-------------------------------------------------------------------------------#
text(7,-3,'By extc PRMITR');
seq_string = ['Input msg sequence = ',num2str(msg_seq)];
text(-4,12,seq_string);
#------------------------------- Bipolar RZ ----- ---------------------------#
%indexx = 1;
%time = 0 ;
%Tb = 1;
%%
BRZ_sarray0 = [0 0 0 0];
%BRZ_sarray1 = [1 1 0 0;-1 -1 0 0];
%shift = [1,2];
%for n = 1:L
% if(msg_seq(n)==0)
% seq_format(indexx:indexx+3) = BRZ_sarray0;
% else
% seq_format(indexx:indexx+3) = BRZ_sarray1(shift(1),:);
% shift = [shift(2),shift(1)];
% endif
% time_format(indexx:indexx+3) = [time time+(Tb/2) time+(Tb/2) time+Tb];
% indexx = indexx + 4;
% time = time + Tb;
%endfor

%subplot(4,2,6),plot(time_format,seq_format,'r'),grid on;
%title("Bipolar RZ"),xlabel("Time --->"),ylabel("Amplitude");
%axis([0 L+1 -1.5 1.5]);
#-------------------------------------------------------------------------------#
```

RESULT:

CONCLUSION:

VIVA QUESTIONS:

1. What is line coding in digital communication?
2. Why is line coding required before digital transmission?
3. What is the difference between unipolar and bipolar line coding?
4. What is DC component and why is it undesirable in line coding?
5. What are the different types of line coding techniques?

EXPERIMENT NO. 6

AIM: To perform Binary Phase Shift Keying (BPSK) modulation and demodulation using a BPSK trainer kit and observe the corresponding waveforms.

Apparatus Required: BPSK Modulation & Demodulation Trainer Kit, CRO / DSO, Patch Cords, Regulated Power Supply

Theory: BPSK Modulation

Binary Phase Shift Keying (BPSK) is a digital modulation technique in which the phase of a sinusoidal carrier is varied in accordance with the binary input data, while the amplitude and frequency of the carrier remain constant.

1. When the input data bit is logic 1, the carrier is transmitted with 0° phase.

2. When the input data bit is logic 0, the carrier is transmitted with a 180° phase shift (phase reversal).

Thus, BPSK uses two phases of the carrier separated by 180° to represent binary data.

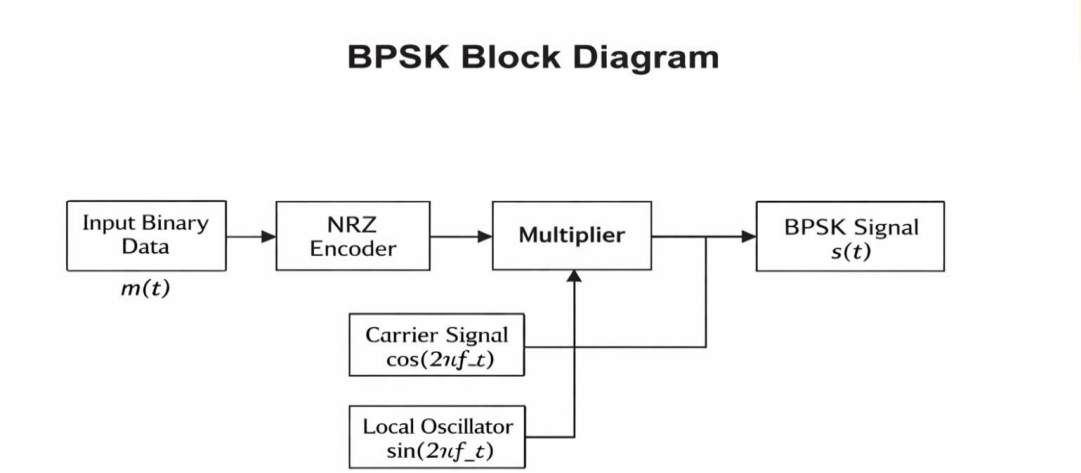
BPSK Demodulation

BPSK demodulation is a coherent detection process, which means the receiver requires a reference carrier that is synchronized in frequency and phase with the transmitted carrier.

1. The received BPSK signal is multiplied with a locally generated reference carrier.
2. The product contains a baseband (data) component and a high-frequency component**.**
3. A Low Pass Filter (LPF) removes the high-frequency component.
4. A comparator or decision circuit determines the transmitted bit:
    - Positive output → logic 1
    - Negative output → logic 0

Thus, the original binary data is recovered.

Block Diagram:



Procedure:

A. BPSK Modulation

1. Switch ON the BPSK trainer kit.
2. Connect the binary data output to the modulator section.
3. Apply the carrier signal from the internal oscillator.
4. Observe the input data, carrier, and BPSK output on CRO.
5. Note the phase change of the carrier for different input bits.

B. BPSK Demodulation

1. Feed the BPSK modulated signal to the demodulator input.
2. Apply the reference carrier to the multiplier.
3. Observe the demodulated signal after the LPF.
4. Compare the recovered binary data with the input data.

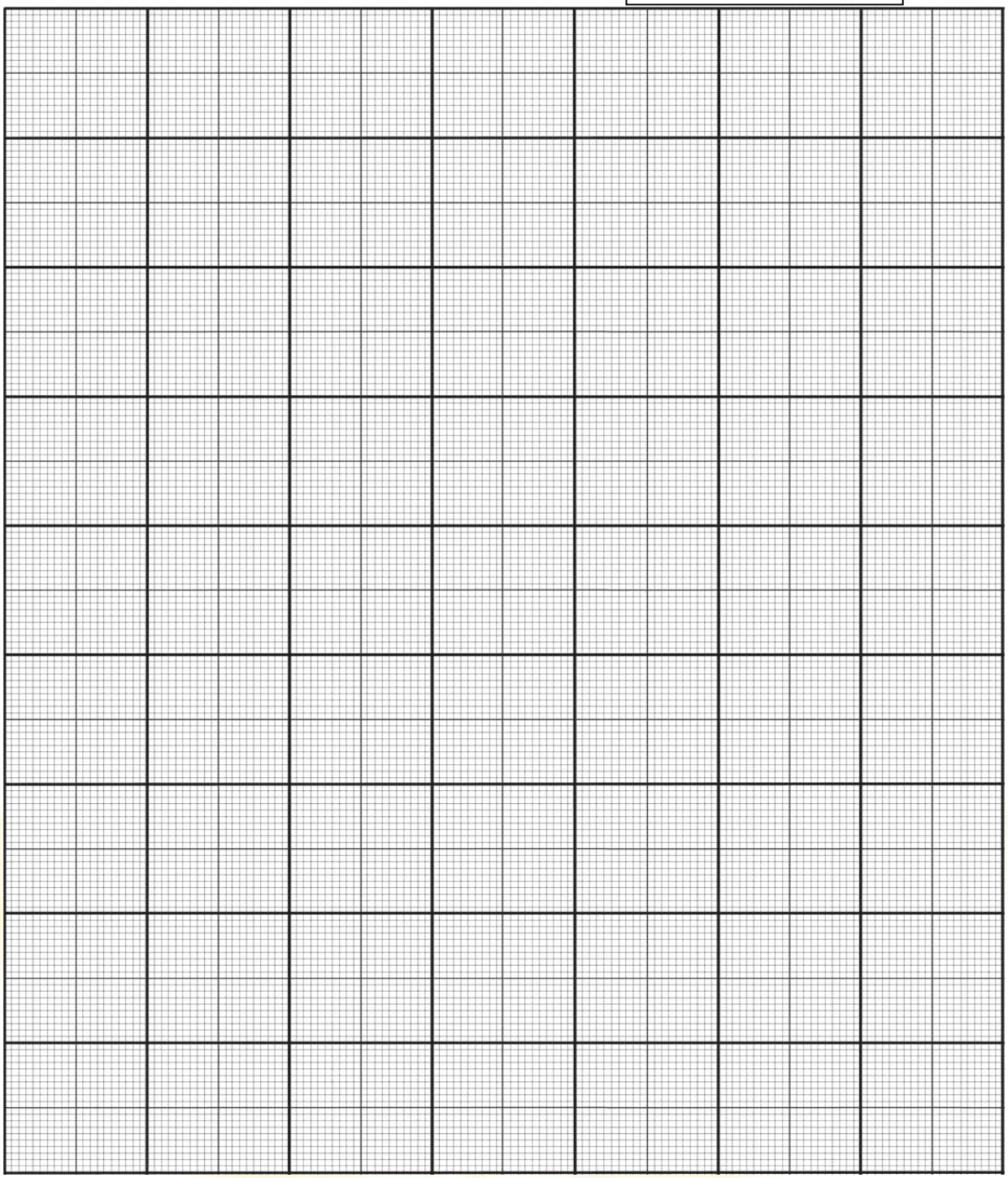
Observation Table:

Sr. No		**Input Bit**	**Carrier Phase**	**BPSK Output**
1	BPSK Modulation			
2				
3				
4				

r. No		**BPSK Input**	**Demodulated Output**	**Output Bit**
1	BPSK Demodulation			
2				
3				
4				

6. **Graph: -BPSK Signal**

Scale
On X-axis 1cm =
On Y-axis 1cm =



RESULT:

CONCLUSION:

VIVA QUESTIONS:

1. What is BPSK?
2. What phase represents binary ‘1’ in BPSK?
3. What phase represents binary ‘0’ in BPSK?
4. Which modulation has better noise immunity: ASK or BPSK?
5. What is the main disadvantage of BPSK?

EXPERIMENT NO. 7

AIM: Develop a program to generate the plot for various basic signals in signal processing.

Software Tool used: Simulator: Octave

Theory: In signal processing, basic signals include unit step, unit impulse, unit ramp, and sinusoidal signals. These signals are fundamental building blocks for analyzing and manipulating other signals and systems. They are often used to characterize the behavior of systems and to test their responses.

Procedure:

1. Open Octave and create a new script file.
2. Clear previous data using commands: clc; clear; close all;
3. Define the discrete time axis as: n = -10 to 10.
4. Unit Impulse Signal: d(n) = 1 for n = 0, and 0 for n ¹ 0.
5. Unit Step Signal: u(n) = 1 for n ³ 0, and 0 for n < 0.
6. Unit Ramp Signal: r(n) = n · u(n).
7. Sinusoidal Signal: x(t) = A sin(2pft + f), where A is amplitude, f is frequency, and f is phase.
8. Exponential Signal: x(n) = e^(a n) · u(n), where 'a' controls growth or decay.
9. Generate each signal using the above expressions in Octave.
10. Plot discrete-time signals using stem() and continuous-time signals using plot().
11. Label the axes, add titles, and enable grid for clarity.
12. Run the program and observe the generated signal waveforms.

Code:

```
clc;
clear;
close all;

% Time axis
n = -10:10;

% 1. Unit Impulse Signal
impulse = (n == 0);

% 2. Unit Step Signal
step = (n >= 0);

% 3. Unit Ramp Signal
ramp = n .* (n >= 0);

% 4. Sinusoidal Signal
t = 0:0.01:1;
sine = sin(2*pi*5*t);

% 5. Exponential Signal
expo = exp(0.5*n) .* (n >= 0);

% Plotting
```

```
figure;

subplot(3,2,1);
stem(n, impulse, 'filled');
xlabel('n'); ylabel('Amplitude');
title('Unit Impulse Signal');
grid on;

subplot(3,2,2);
stem(n, step, 'filled');
xlabel('n'); ylabel('Amplitude');
title('Unit Step Signal');
grid on;

subplot(3,2,3);
stem(n, ramp, 'filled');
xlabel('n'); ylabel('Amplitude');
title('Unit Ramp Signal');
grid on;

subplot(3,2,4);
plot(t, sine);
xlabel('Time'); ylabel('Amplitude');
title('Sinusoidal Signal');
grid on;

subplot(3,2,5);
stem(n, expo, 'filled');
xlabel('n'); ylabel('Amplitude');
title('Exponential Signal');
grid on;
```

RESULT:

CONCLUSION:

VIVA QUESTIONS:

1. What are basic signals in signal processing?
2. How do you generate a unit impulse signal in code?
3. Explain plotting a sinusoidal signal.
4. What is the difference between continuous and discrete signals?
5. How to customize plots for better visualization?

EXPERIMENT NO. 8

AIM: Develop a program for the convolution of two discrete time signals using the Matrix method and validate the result using an in-built convolution command.

Software Tool used: Simulator: Octave

Theory:

The convolution of two discrete-time signals, x[n] and h[n], results in a new signal, y[n], which is calculated by summing the products of one signal with a time-reversed and shifted version of the other. This operation is fundamental in signal processing, especially for analyzing linear time-invariant (LTI) systems.

$$y(n) = x(n) * h(n)$$

Procedure:

1. Define two discrete-time signals x(n) and h(n)
2. Determine lengths of the signals
3. Construct a convolution matrix from x(n)
4. Multiply the matrix with h(n)
5. Obtain convolution output using matrix method
6. Validate the result using Octave built-in conv()
7. Compare both outputs

Code:

```
clc;
clear;
close all;

% Input Signals
x = [1 2 3];        % First signal x(n)
h = [2 1];          % Second signal h(n)

Lx = length(x);
Lh = length(h);

% Matrix Method
% Create convolution matrix
X = zeros(Lx+Lh-1, Lh);

for i = 1:Lh
    X(i:i+Lx-1, i) = x';
end

% Convert h to column vector
```

```
h_col = h';

% Convolution using matrix method
y_matrix = X * h_col;

% Built-in Convolution
y_builtin = conv(x, h);

% Display Results
disp('Convolution using Matrix Method:');
disp(y_matrix');

disp('Convolution using Built-in conv():');
disp(y_builtin);

% Plot Results
n = 0:length(y_builtin)-1;

figure;
subplot(3,1,1)
stem(0:Lx-1, x, 'filled');
xlabel('n'); ylabel('Amplitude');
title('Input Signal x(n)');
grid on;

subplot(3,1,2)
stem(0:Lh-1, h, 'filled');
xlabel('n'); ylabel('Amplitude');
title('Impulse Response h(n)');
grid on;

subplot(3,1,3)
stem(n, y_builtin, 'filled');
xlabel('n'); ylabel('Amplitude');
title('Convolution Output y(n)');
grid on;
```

RESULT:

CONCLUSION:

VIVA QUESTIONS

1. What is convolution in discrete-time signal processing?
2. Why is the matrix method used to perform convolution?
3. What is the purpose of the in-built conv() function in Octave/MATLAB?
4. How do you validate the matrix method convolution result?
5. What is the length of the output signal after convolution?

EXPERIMENT NO. 9

AIM: To develop a program to determine the linearity and time invariance properties of a discrete-time system.

Software Tool used: Simulator: Octave

Theory:
Linearity and time invariance are important properties of discrete-time systems used in signal processing.

Linearity:
A system is linear if it satisfies the principle of superposition.
T[a x1(n) + b x2(n)] = a T[x1(n)] + b T[x2(n)]

Time Invariance:
A system is time invariant if a time shift in the input causes an identical time shift in the output.
If x(n) →y(n), then x(n-n0) →y(n-n0)
System Considered
y(n) = 2x(n)

Procedure:
1. Define two discrete-time input signals.
2. Test linearity using scaling and addition.
3. Delay the input signal.
4. Compare output of delayed input with delayed output.
5. Verify system properties.

Code:
```
clc;
clear;
close all;

% Define signals
n = -5:5;
x1 = sin(n);
x2 = cos(n);

a = 2;
b = 3;

% System definition
% y(n) = 2x(n)
T = @(x) 2*x;

% Linearity Test
lhs = T(a*x1 + b*x2);
```

```
rhs = a*T(x1) + b*T(x2);

% Time Invariance Test
n0 = 2;
x = sin(n);

y = T(x);                % Original output
x_delayed = sin(n-n0);    % Delayed input
y1 = T(x_delayed);        % Output for delayed input
y2 = circshift(y, [0 n0]);% Delayed output

% Display Results
disp('Linearity Test (LHS):');
disp(lhs);

disp('Linearity Test (RHS):');
disp(rhs);

disp('Time Invariance Test (Output of delayed input):');
disp(y1);

disp('Time Invariance Test (Delayed output):');
disp(y2);

% Plotting
figure;

subplot(2,1,1)
stem(n, lhs, 'filled');
title('Linearity Test Output');
xlabel('n'); ylabel('Amplitude');
grid on;

subplot(2,1,2)
stem(n, y1, 'filled');
title('Time Invariance Test Output');
xlabel('n'); ylabel('Amplitude');
grid on;
```

RESULT:

CONCLUSION:

VIVA QUESTIONS

1. What is superposition and how to check linearity in code?
2. How to verify time-invariance using delayed inputs?
3. For y[n] = n*x[n], is it linear? How to test in program?
4. How to create test inputs like impulses for system checks?
5. What issues arise with floating-point in property checks?

EXPERIMENT NO. 10

AIM: Develop program to determine the Fourier Transform of Discrete Time Signal and to plot magnitude and Phase response.

Software Tool used: Simulator: Octave

Theory:

Procedure:

1. Open GNU Octave and create a new script file to write the program.
2. Clear the command window, workspace variables, and close all previously opened figures using the commands clc, clear all, and close all.
3. Define the frequency range ω from −π to π with a step size of 0.01 radians.
4. Define the discrete-time index n from 0 to 50 and generate the discrete-time signal

$$x(n) = (0.5)^n + (-0.3)^n$$

5. Compute the Discrete-Time Fourier Transform (DTFT) manually using nested for loops by evaluating the DTFT summation formula for each frequency value.
6. Plot the magnitude response of the DTFT using abs(X) and label the axes appropriately.
7. Plot the phase response of the DTFT using arg(X) and label the axes appropriately.
8. Use the built-in DTFT function to compute the Fourier Transform of the signal for verification.
9. Plot the magnitude response obtained using the DTFT function and label the axes.
10. Plot the phase response obtained using the DTFT function and compare it with the manually computed result.
11. Observe that both methods produce similar magnitude and phase responses, thus verifying the correctness of the DTFT computation.

Code:

```
clc;
clear all;
close all;
w = -pi:0.01:pi;
n=0:50;
x= (0.5).^n + (-0.3).^n;
for ii = 1:length(w)
    X(ii) =0;
    for kk=1:length(x)
    X(ii)= X(ii) + x(kk).* (exp(-1j.*w(ii).*n(kk)));
  endfor
endfor

subplot(221)
plot(w,abs(X));
xlabel('omega(w)')
ylabel('Magnitude')
```

```
title('Magnitude Response')
grid on

subplot(222)
plot(w,arg(X));
xlabel('omega(w)')
ylabel('Argument / Phase')
title('Phase Response')
grid on

[z] = dtft(x,n,w);

subplot(223)
plot(w,abs(z));
xlabel('omega(w)')
ylabel('Magnitude')
title('Magnitude Response using DTFT function')
grid on

subplot(224)
plot(w,arg(z));
xlabel('omega(w)')
ylabel('Argument / Phase')
title('Phase Response using DTFT function')
grid on
```

RESULT:

CONCLUSION:

VIVA QUESTIONS

1. What is the Fourier Transform of a discrete-time signal called?
2. Why do we plot magnitude and phase response of a signal?
3. What is the role of FFT in Fourier Transform computation?
4. What information does the magnitude spectrum represent?
5. Why is phase response important in signal reconstruction?

EXPERIMENT NO. 11

AIM: Develop a program to verify the following properties of DTFT

i)Time Shifting Property ii) Frequency Shifting Property

Software Tool used: Simulator: Octave

Theory:

The Discrete-Time Fourier Transform (DTFT) possesses several key properties that are crucial for analysing and manipulating discrete-time signals. These include linearity, time shifting, frequency shifting, time reversal, conjugation, and time scaling

Procedure:

1. Open GNU Octave and create a new script file (.m file).
2. Clear the command window, workspace, and close all figures using clc, clear all, and close all.
3. Define the discrete-time index n and generate the original discrete-time signal x(n).
4. Define the frequency range ω from −π to π with a small step size.
5. Compute the DTFT of the original signal using the DTFT summation formula inside nested for loops.
6. For Time Shifting Property:
   - Shift the signal by n_0 samples to obtain $x(n - n_0)$.
   - Compute the DTFT of the time-shifted signal.
   - Compute the theoretical DTFT by multiplying the original DTFT with $e^{(-j\omega n_0)}$.
7. For Frequency Shifting Property:
   - Multiply the original signal with a complex exponential $e^{(j\omega_0 n)}$ to obtain the frequency-shifted signal.
   - Compute the DTFT of the frequency-shifted signal.
8. Plot the magnitude responses of:
   - Original DTFT
   - Time-shifted DTFT (computed and theoretical)
   - Frequency-shifted DTFT
9. Compare the plots to verify the properties:
   - Time shifting results in a phase change without affecting magnitude.
   - Frequency shifting causes a shift in the spectrum along the frequency axis.
10. Save the script and execute it to verify the DTFT properties.

Code:

```
clc;
clear all;
close all;

% Discrete-time signal
n = 0:20;
x = (0.8).^n;          % Original signal
```

```
w = -pi:0.01:pi;

% DTFT of original signal
X = zeros(1,length(w));
for ii = 1:length(w)
   for kk = 1:length(n)
      X(ii) = X(ii) + x(kk)*exp(-1j*w(ii)*n(kk));
   end
end

% Time Shifting Property
n0 = 5;
x_shift = (0.8).^(n-n0).*(n>=n0);   % Time shifted signal

X_shift = zeros(1,length(w));
for ii = 1:length(w)
   for kk = 1:length(n)
      X_shift(ii) = X_shift(ii) + x_shift(kk)*exp(-1j*w(ii)*n(kk));
   end
end

X_time_theory = X .* exp(-1j*w*n0);

% Frequency Shifting Property
w0 = pi/4;
x_freq = x .* exp(1j*w0*n);

X_freq = zeros(1,length(w));
for ii = 1:length(w)
   for kk = 1:length(n)
      X_freq(ii) = X_freq(ii) + x_freq(kk)*exp(-1j*w(ii)*n(kk));
   end
end

% Plots
figure;
subplot(3,1,1);
plot(w, abs(X));
title('Magnitude of Original DTFT');
xlabel('\omega'); ylabel('|X(e^{j\omega})|'); grid on;

subplot(3,1,2);
plot(w, abs(X_shift), 'r', w, abs(X_time_theory), '--k');
title('Time Shifting Verification');
legend('Computed','Theoretical');
```

```
xlabel('\omega'); ylabel('Magnitude'); grid on;

subplot(3,1,3);
plot(w, abs(X_freq));
title('Frequency Shifting Verification');
xlabel('\omega'); ylabel('Magnitude'); grid on;
```

RESULT:

CONCLUSION:

VIVA QUESTIONS

1. Explain how to implement time shifting on a discrete signal in Python to verify the DTFT Time Shifting property.
2. Describe the steps to numerically compute the DTFT of a shifted signal and compare it with the theoretical result.
3. How would you modulate a signal with a complex exponential in code to test the Frequency Shifting property of DTFT?
4. What test signal (e.g., impulse or sinusoid) would you use in a program to demonstrate both DTFT shifting properties, and why?
5. Outline the Python code structure to plot and verify the magnitude and phase differences due to frequency shifting in DTFT.

EXPERIMENT NO. 12

AIM: Develop a program to determine Z-Transform of a Discrete Time signal.

Software Tool used: Octave

Theory:

The Z-Transform is a mathematical tool used to analyse discrete-time signals and systems in the complex frequency domain.
It converts a time-domain sequence x(n) into a complex function X(z).

Z-Transform is widely used to:

- Analyse stability and causality of systems
- Solve difference equations
- Design and analyse digital filters

For practical implementation, the Z-Transform of a finite-length sequence is computed using summation in Octave.

Procedure:

1. Start the Octave software.
2. Clear the command window, workspace, and close all figures.
3. Define the discrete-time signal x(n).
4. Define the complex variable z.
5. Apply the Z-Transform formula using a loop or vector operation.
6. Display the symbolic or numerical Z-Transform result.
7. Verify the output.

Code:

```
clc;
clear;
close all;

% Load symbolic package (required in Octave)
pkg load symbolic

% Define discrete-time signal
x = [1 2 3];          % x(n)
n = 0:length(x)-1;   % Time index

% Define Z variable
syms z

% Z-Transform computation
Xz = 0;
for k = 1:length(x)
```

```
    Xz = Xz + x(k)*z^(-n(k));
end

% Display result
disp('Z-Transform of the given sequence is:');
pretty(Xz)
```

RESULT:

CONCLUSION:

VIVA QUESTIONS

1. What is the Z-transform of a discrete-time signal and why is it used in signal processing?
2. How is the time index nnn defined for a discrete-time sequence in Octave?
3. What is the role of the symbolic variable z in the Octave program?
4. How does the region of convergence (ROC) affect the Z-transform of a sequence?